



Chapter 1: Introduction

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan

See www.db-book.com for conditions on re-use



Anahat

- Veritabanı-Sistem Uygulamaları
- Veritabanı Sistemlerinin Amacı
- Veriye Bakış
- Veritabanı Dilleri
- Veritabanı Tasarımı
- Veritabanı Motoru
- Veritabanı Mimarisi
- Veritabanı Kullanıcıları ve Yöneticileri
- Veritabanı Sistemlerinin Tarihçesi



Veritabanı Sistemleri

- Veritabanı yönetim sistemi (VTYS) belirli bir işletme hakkında bilgi içerir
 - Birbiriyle ilişkili veriler topluluğu
 - Verilere erişmek için programlar kümesi
 - Kullanımı hem uygun hem de verimli bir ortam
- Veritabanı sistemleri aşağıdaki veri koleksiyonlarını yönetmek için kullanılır:
 - Çok değerli
 - Nispeten büyük
 - Genellikle aynı anda birden fazla kullanıcı ve uygulama tarafından erişilir
- Modern bir veritabanı sistemi, görevi büyük ve karmaşık bir veri koleksiyonunu yönetmek olan karmaşık bir yazılım sistemidir
- Veritabanları hayatımızın her alanına dokunur



Veritabanı Uygulamaları Örnekleri

- Kurumsal Bilgi
 - Satış: müşteriler, ürünler, satın alımlar
 - Muhasebe: ödemeler, makbuzlar, varlıklar
 - İnsan Kaynakları: Çalışanlar, maaşlar, bordro vergileri hakkında bilgiler
- Üretim: üretim, envanter, siparişler, tedarik zinciri yönetimi
- Bankacılık ve finans
 - müşteri bilgileri, hesaplar, krediler ve bankacılık işlemleri
 - Kredi kartı işlemleri
 - Finans: finansal araçların satış ve alımları (ör. hisse senetleri ve tahviller; gerçek zamanlı piyasa verilerinin depolanması)
- Üniversiteler: kayıt, notlar



Veritabanı Uygulamaları Örnekleri

- Havayolları: rezervasyonlar, tarifeler
- Telekomünikasyon: aramaların, metinlerin ve veri kullanımının kayıtları, aylık faturaların oluşturulması, ön ödemeli arama kartlarındaki bakiyelerin korunması
- Web tabanlı hizmetler
 - Sosyal medya
 - Çevrimiçi perakendeciler: sipariş takibi, özelleştirilmiş öneriler
 - Çevrimiçi reklamlar
- Belge veritabanları
- Navigasyon sistemleri: Yolların, tren sistemlerinin, otobüslerin vb. kesin rotaları ile birlikte çeşitli ilgi çekici yerlerin konumlarını korumak için.



Veritabanı Sistemlerinin Amacı

İlk zamanlarda, veritabanı uygulamaları doğrudan dosya sistemlerinin üzerine inşa ediliyordu. Bu durumun sonuçları:

- Veri fazlalığı ve tutarsızlığı: verilerin birden fazla dosya formatında saklanması ve bunun sonucunda farklı dosyalardaki bilgilerin tekrarlanması
- Verilere erişim zorluğu
 - Her yeni görevi yerine getirmek için yeni bir program yazma ihtiyacı
- Veri izolasyonu
 - Birden fazla dosya ve format
- Bütünlük sorunları
 - Bütünlük kısıtlamaları (örneğin, hesap bakiyesi > 0) açıkça belirtilmek yerine program koduna "gömülür"
 - Yeni kısıtlamalar eklemek veya mevcut olanları değiştirmek zordur



Veritabanı Sistemlerinin Amacı

- Güncellemelerin atomikliği
 - Hatalar, yapılan kısmi güncellemelerle veritabanını tutarsız bir durumda bırakabilir
 - Örnek: Bir hesaptan diğerine para transferi ya tamamlanmalı ya da hiç gerçekleşmemelidir
- Birden fazla kullanıcı tarafından eşzamanlı erişim
 - Performans için eşzamanlı erişim gereklidir
 - Kontrolsüz eşzamanlı erişimler tutarsızlıklara yol açabilir
 - Örn: İki kişinin aynı anda bir bakiyeyi (örneğin 100) okuması ve para çekerek (örneğin her biri 50) güncellemesi
- Güvenlik sorunları
 - Veritabanı sisteminin her kullanıcısı tüm verilere erişememelidir.

Veritabanı sistemleri yukarıdaki tüm sorunlara çözüm sunar



Üniversite Veritabanı Örneği

- Tüm kavramları göstermek için bir üniversite veritabanı kullanacağız
- Veriler aşağıdakiler hakkında bilgilerden oluşur:
 - Öğrenciler
 - Öğretmenler
 - Sınıflar
- Uygulama programı örnekleri:
 - Yeni öğrenciler, öğretmenler ve dersler ekleme
 - Öğrencileri derslere kaydetme ve sınıf listeleri oluşturma
 - Öğrencilere not atama, not ortalamalarını hesaplama ve transkript oluşturma



Veri Görünümü

- Bir veritabanı sistemi, birbiriyle ilişkili veriler ve kullanıcıların bu verilere erişmesine ve bunları değiştirmesine olanak tanıyan bir dizi programdan oluşan bir koleksiyondur.
- Bir veritabanı sisteminin **temel amacı**, kullanıcılara verilerin **soyut bir görünümünü** sağlamaktır.
 - Veri modelleri
 - Verileri, veri ilişkilerini, veri semantiğini ve tutarlılık kısıtlamalarını tanımlamak için kavramsal araçlar koleksiyonu.
 - Veri soyutlama
 - Veritabanındaki verileri temsil etmek için veri yapılarının karmaşıklığını, çeşitli veri soyutlama seviyeleri aracılığıyla kullanıcılardan gizler.



Veri Modelleri

Veri modelleri şunları tanımlama bir araç koleksiyonudur:

- Veri
- Veri ilişkileri
- Veri anlambilimi
- Veri kısıtlamaları
- İlişkisel model
 - Varlık-İlişki veri modeli (esas olarak veritabanı tasarımı için)
 - Nesne tabanlı veri modelleri (Nesne yönelimli ve Nesne-ilişkisel)
- Yarı yapılandırılmış veri modeli (XML,JSON,CSV)
- Nesne tabanlı veri modeli:
 - Sınıflar tablolarla
 - Nesneler verilerle
 - ORM (object relational mapper)
 - Entity framework
 - Hibernate



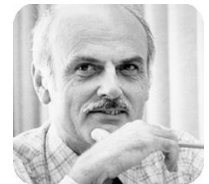
İlişkisel Model

- Tüm veriler çeşitli tablolarda saklanır.
- İlişkisel modelde tablo verisi örneği

Columns

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows



Ted Codd
Turing Award 1981

(a) The *instructor* table



Örnek Bir İlişkisel Veritabanı

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table



Soyutlama Düzeyleri

- **Fiziksel seviye:** bir kaydın (örneğin öğretmen) nasıl saklandığını açıklar.
- **Mantıksal seviye:** veritabanında depolanan verileri ve veriler arasındaki ilişkileri tanımlar.

```
type instructor = record
```

```
    ID : string;
```

```
    name : string;
```

```
    dept_name : string;
```

```
    salary : integer;
```

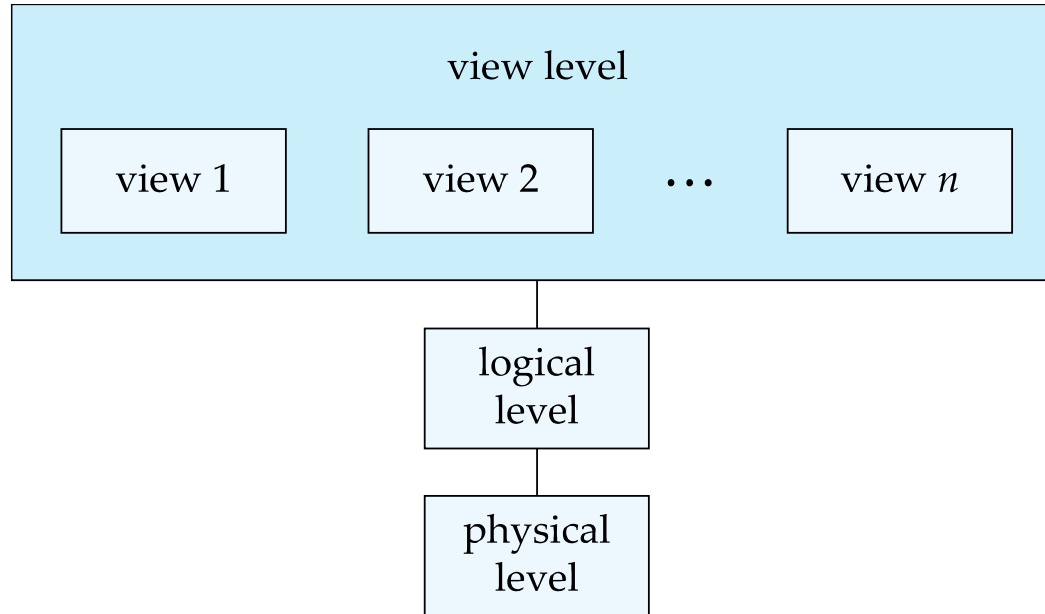
```
end;
```

- **Görünüm seviyesi:** kullanıcılar bütün veriler yerine ilgilendikleri özet verileri görmek isterler. Sorgular sayesinde veri özetleri elde edilebilir.



Veri Görünümü

Veritabanı sistemi için bir mimari





Instances and Schemas

- Similar to types and variables in programming languages
- **Logical Schema** – the overall logical structure of the database
 - Example: The database consists of information about a set of customers and accounts in a bank and the relationship between them
 - Analogous to type information of a variable in a program
- **Physical schema** – the overall physical structure of the database
- **Instance** – the actual content of the database at a particular point in time
 - Analogous to the value of a variable



Fiziksel Veri Bağımsızlığı

- **Fiziksel Veri Bağımsızlığı**– mantıksal şemayı değiştirmeden fiziksel şemayı değiştirme yeteneği
 - Uygulamalar mantıksal şemaya bağlıdır
 - Genel olarak, çeşitli düzeyler ve bileşenler arasındaki arayüzler, bazı bölümlerdeki değişikliklerin diğerlerini ciddi şekilde etkilememesi için iyi tanımlanmalıdır.



Data Definition Language (DDL)

- Veritabanı şemasını tanımlamak için spesifikasyon notasyonu

Örnek: **create table** *instructor* (
 ID **char**(5),
 name **varchar**(20),
 dept_name **varchar**(20),
 salary **numeric**(8,2))

- DDL derleyicisi, bir **veri sözlüğünde** depolanan bir dizi tablo şablonu oluşturur
- Veri sözlüğü meta verileri (yani veriler hakkındaki verileri) içerir
 - Veritabanı şeması
 - Bütünlük kısıtlamaları
 - Birincil anahtar (eğitmenleri benzersiz olarak tanımlayan ID)
 - Yetkilendirme
 - Kim neye erişebilir



Data Manipulation Language (DML)

- Uygun veri modeli tarafından düzenlenen verilere erişmek ve bunları güncellemek için kullanılan dil
 - DML, sorgu dili olarak da bilinir
- Temel olarak iki tür veri manipülasyon dili vardır
 - **Procedural DML** -- kullanıcının hangi verilere ihtiyaç duyulduğunu ve bu verilerin nasıl elde edileceğini belirtmesini gerektirir.
 - **Declarative DML** -- kullanıcının hangi verilere ihtiyaç duyulduğunu belirtmesini, ancak bu verilerin nasıl elde edileceğini belirtmemesini gerektirir.
- Declarative DML'lerin öğrenilmesi ve kullanılması genellikle prosedürel DML'lere göre daha kolaydır.
- Declarative DML'ler aynı zamanda prosedürel olmayan DML'ler olarak da adlandırılır
- DML'nin bilgi erişimini içeren kısmına sorgu dili denir.



SQL Sorgu Dili

- SQL sorgu dili prosedürel değildir.
- Bir sorgu girdi olarak birkaç tablo alır (muhtemelen sadece bir tane) ve her zaman tek bir tablo döndürür.
- Comp.Sci bölümündeki tüm eğitmenleri bulmak için örnek

```
select name
from instructor
where dept_name = 'Comp. Sci.'
```
- SQL, Turing makinesine eşdeğer bir dil DEĞİLDİR
- Karmaşık fonksiyonları hesaplayabilmek için SQL genellikle bazı üst düzey dillere gömülüdür
- Uygulama programları genellikle veritabanlarına şu dil uzantılarından biri aracılığıyla erişir
 - Gömülü SQL'e izin vermek için dil uzantıları
 - sorgularının bir veritabanına gönderilmesine izin veren uygulama programı arayüzü (örneğin, ODBC/JDBC)



Uygulama Programından Veritabanı Erişimi

- SQL gibi prosedürel olmayan sorgu dilleri evrensel bir Turing makinesi kadar güçlü değildir.
- SQL, **kullanıcılardan gelen girdi, ekranlara çıktı veya ağ üzerinden iletişim** gibi eylemleri **desteklemez**.
- Bu tür hesaplamalar ve eylemler, C/C++, Java veya Python gibi bir ana dilde, veritabanındaki verilere erişen gömülü SQL sorguları ile yazılmalıdır.
- **Application programs** -- veritabanı ile bu şekilde etkileşim kurmak için kullanılan programlardır.



Veritabanı Tasarımı

Veritabanının genel yapısını tasarlama süreci:

- Mantıksal Tasarım - Veritabanı şemasına karar verme. Veritabanı tasarımı, "iyi" bir ilişki şemaları koleksiyonu bulmamızı gerektirir.
 - İş kararı - Veritabanına hangi nitelikleri kaydetmeliyiz?
 - Bilgisayar Bilimi kararı - Hangi ilişki şemalarına sahip olmalıyız ve nitelikler çeşitli ilişki şemaları arasında nasıl dağıtılmalıdır?
- Fiziksel Tasarım - Veritabanının fiziksel düzenine karar vermek
 - Tek sunucu
 - Dağıtık sistem



Veritabanı Motoru

- Bir veritabanı sistemi, genel sistemin sorumluluklarının her biriyle ilgilenen modüllere ayrılır.
- Bir veritabanı sisteminin işlevsel bileşenleri
 - Depolama yöneticisi,
 - Sorgu işlemcisi bileşeni,
 - Transaction yönetimi bileşeniolarak ayrılabilir.



Depolama Yöneticisi

- Veritabanında depolanan düşük seviyeli veriler ile uygulama programları ve sisteme gönderilen sorgular arasında arayüz sağlayan bir program modülü.
- Depolama yöneticisi aşağıdaki görevlerden sorumludur:
 - İşletim sistemi dosya yöneticisi ile etkileşim
 - Verilerin verimli bir şekilde saklanması, alınması ve güncellenmesi
- Depolama yöneticisi bileşenleri şunları içerir:
 - Yetkilendirme ve bütünlük yöneticisi
 - İşlem yöneticisi
 - Dosya yöneticisi
 - Arabellek yöneticisi



Depolama Yöneticisi

- Depolama yöneticisi, fiziksel sistem uygulamasının bir parçası olarak çeşitli veri yapıları uygular :
 - Veri dosyaları -- veritabanının kendisini depolar
 - Veri sözlüğü -- veritabanının yapısı, özellikle de veritabanının şeması hakkında meta verileri depolar
 - İndeksler -- veri öğelerine hızlı erişim sağlayabilir. Bir veritabanı indeksi, belirli bir değeri tutan veri öğelerine işaretçiler sağlar.



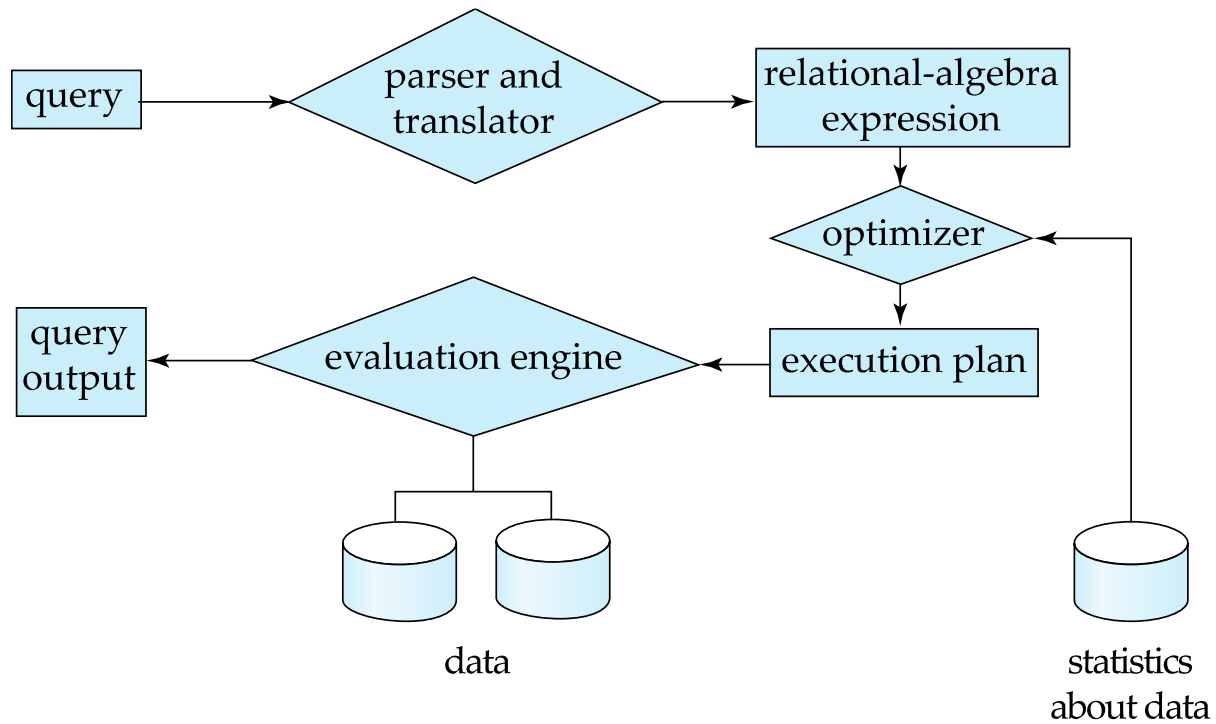
Sorgu İşlemcisi

- Sorgu işlemcisi bileşenleri şunları içerir :
 - DDL interpreter -- DDL ifadelerini yorumlar ve tanımları veri sözlüğüne kaydeder.
 - DML compiler -- Bir sorgu dilindeki DML ifadelerini, sorgu değerlendirme motorunun anlayabileceği düşük seviyeli talimatlardan oluşan bir değerlendirme planına çevirir.
 - DML derleyicisi sorgu optimizasyonunu gerçekleştirir; yani çeşitli alternatifler arasından en düşük maliyetli değerlendirme planını seçer.
 - Query evaluation engine -- DML derleyicisi tarafından oluşturulan düşük seviyeli talimatları yürütür.



Sorgu İşleme

1. Ayırıştırma ve çeviri
2. Optimizasyon
3. Değerlendirme





Transaction Yönetimi

- **transaction**, bir veritabanı uygulamasında tek bir mantıksal işlevi yerine getiren işlemler topluluğudur
- **transaction yönetimi bileşeni**, sistem arızalarına (örn. güç arızaları ve işletim sistemi çökmeleri) ve işlem arızalarına rağmen veritabanının tutarlı (doğru) bir durumda kalmasını sağlar
- **Eşzamanlılık kontrol yöneticisi**, veritabanının tutarlılığını sağlamak için eşzamanlı işlemler arasındaki etkileşimi kontrol eder.

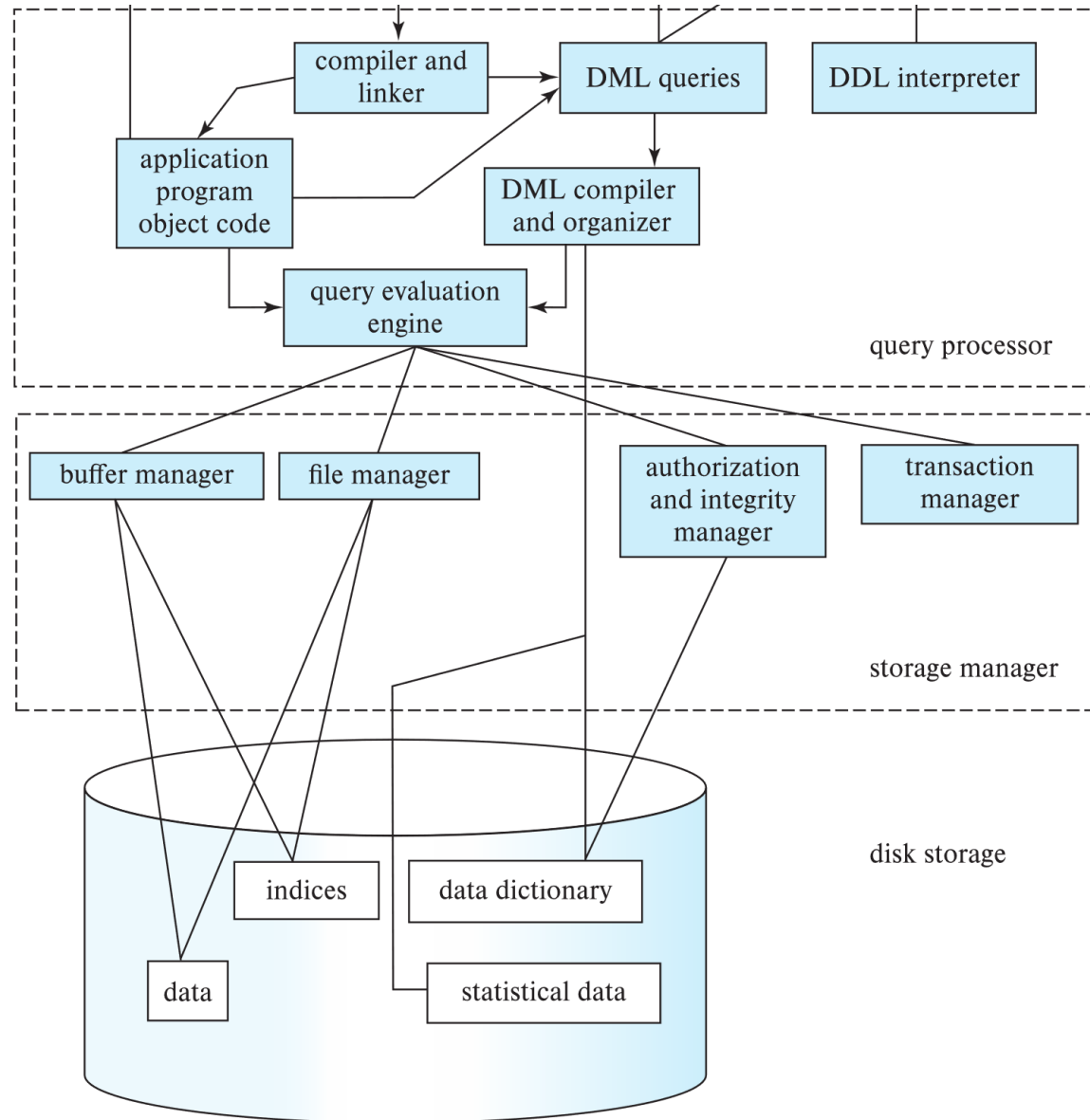


Veritabanı Mimarisi

- Merkezi veritabanları
 - Bir ila birkaç çekirdek, paylaşılan bellek
- İstemci-sunucu,
 - Bir sunucu makine birden fazla istemci makine adına iş yürütür.
- Paralel veritabanları
 - Birçok çekirdek paylaşılan bellek
 - Paylaşılan disk
 - Paylaşılan hiçbir şey
- Dağıtılmış veritabanları
 - Coğrafi dağılım
 - Şema/veri heterojenliği



Veritabanı Mimarisi (Merkezi / Paylaşımlı Bellek)





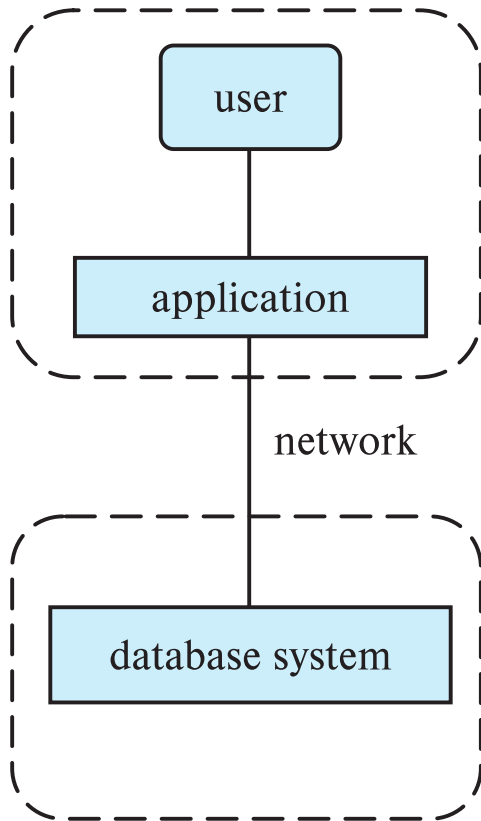
Veritabanı Uygulamaları

Veritabanı uygulamaları genellikle iki veya üç bölüme ayrılır

- İki katmanlı mimari -- uygulama, sunucu makinede veritabanı sistemi işlevselliğini çağırdığı istemci makinede bulunur
- Üç katmanlı mimari -- istemci makine bir ön uç görevi görür ve herhangi bir doğrudan veritabanı çağrısı içermez.
 - İstemci uç, genellikle bir form arayüzü aracılığıyla bir uygulama sunucusuyla iletişim kurar.
 - Uygulama sunucusu da verilere erişmek için bir veritabanı sistemiyle iletişim kurar.



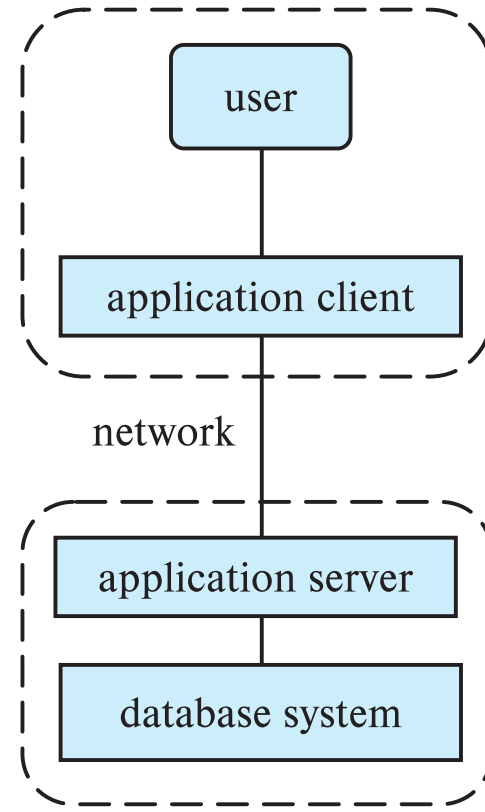
Two-tier and three-tier architectures



(a) Two-tier architecture

client

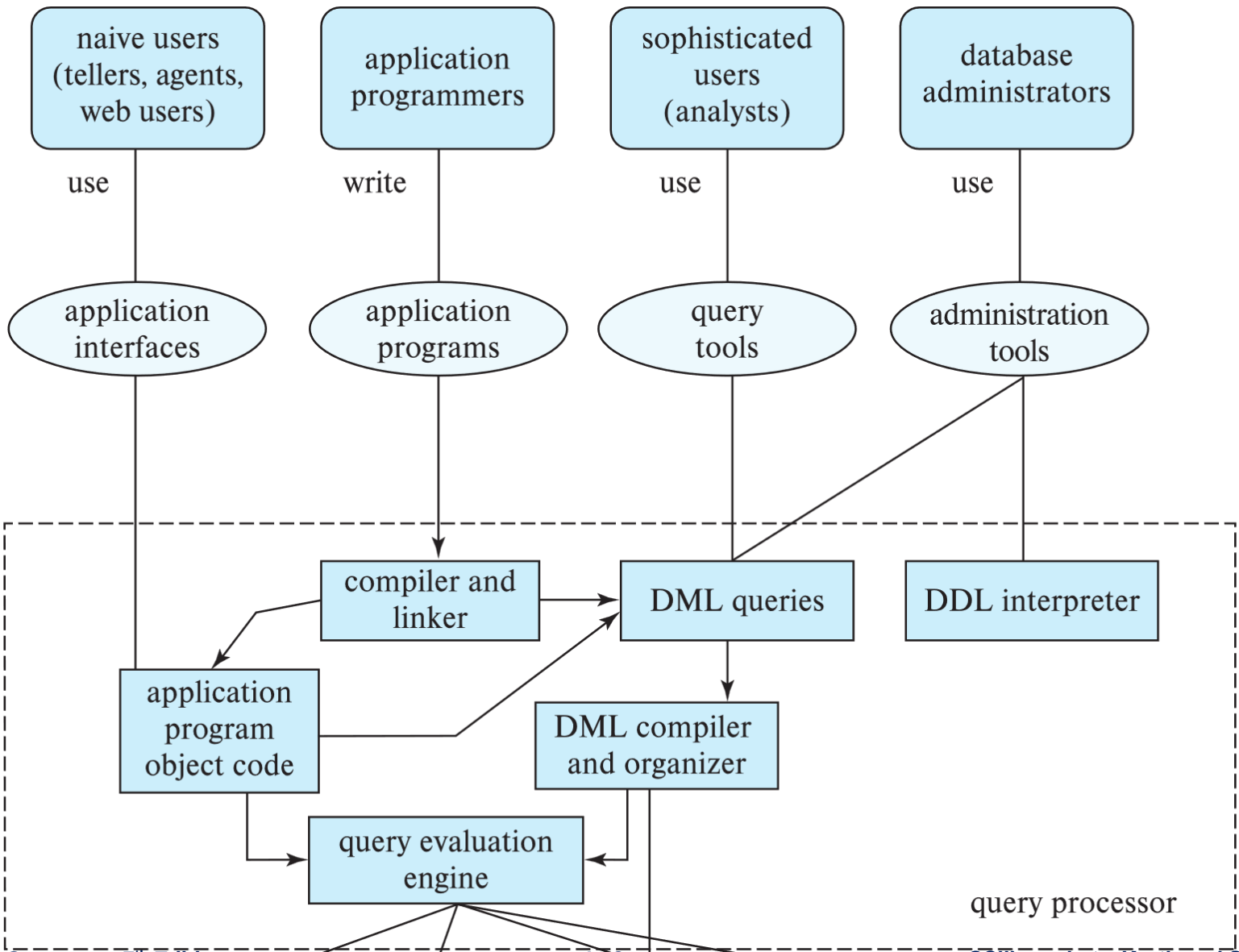
server



(b) Three-tier architecture



Veritabanı Kullanıcıları





Veritabanı Yöneticisi

Sistem üzerinde merkezi kontrole sahip olan kişiye **veritabanı yöneticisi (DBA)** denir. Bir DBA'nın işlevleri şunları içerir:

- Şema tanımı
- Depolama yapısı ve erişim yöntemi tanımı
- Şema ve fiziksel organizasyon değişikliği
- Veri erişimi için yetki verilmesi
- Rutin bakım
- Periyodik olarak veritabanının yedeklenmesi
- Normal işlemler için yeterli boş disk alanının mevcut olduğundan emin olunması ve gerektiğinde disk alanının yükseltilmesi
- Veritabanında çalışan işlerin izlenmesi



Veritabanı Sistemlerinin Tarihçesi

- 1950'ler ve 1960'ların başı:
 - Depolama için manyetik bantlar kullanarak veri işleme
 - Bantlar yalnızca sıralı erişim sağladı
 - Giriş için delikli kartlar
- 1960'ların sonu ve 1970'ler:
 - Sabit diskler verilere doğrudan erişim sağladı
 - Yaygın kullanımda ağ ve hiyerarşik veri modelleri
 - Ted Codd ilişkisel veri modelini tanımladı
 - Bu çalışmasıyla ACM Turing Ödülü'nü kazanacak
 - IBM Research System R prototipini başlattı
 - UC Berkeley (Michael Stonebraker) Ingres prototipini başlattı
 - Oracle ilk ticari ilişkisel veritabanını yayınladı
 - Yüksek performanslı (o dönem için) transaction işleme



Veritabanı Sistemlerinin Tarihçesi

- 1980'ler:
 - Araştırma ilişkisel prototipleri ticari sistemlere dönüşüyor
 - SQL endüstriyel standart haline geliyor
 - Paralel ve dağıtık veritabanı sistemleri
 - Wisconsin, IBM, Teradata
 - Nesne yönelimli veritabanı sistemleri
- 1990'lar:
 - Büyük karar destek ve veri madenciliği uygulamaları
 - Büyük çok terabaytlı veri ambarları
 - Web ticaretinin ortaya çıkışı



Veritabanı Sistemlerinin Tarihçesi

- 2000'ler
 - Büyük veri depolama sistemleri
 - Google BigTable, Yahoo PNuts, Amazon,
 - "NoSQL" sistemleri
 - Büyük veri analizi: SQL'in ötesinde
 - Map reduce ve arkadaşları
- 2010'lar
 - SQL yeniden yüklendi
 - Map Reduce sistemlerine SQL ön ucu
 - Büyük ölçüde paralel veritabanı sistemleri
 - Çok çekirdekli ana bellek veritabanları



Bölüm 1'in Sonu